

IME 775 — Lecture 22

Latent Spaces, PCA, and Autoencoders

Chapter 14 of Math & Architectures of Deep Learning — Part 1 of 2

1. Motivation: The Space of Natural Signals

A digital image of height H and width W with 24-bit RGB colour is, in principle, a point in a space of size $2^{24 HW}$. For a 28×28 MNIST digit alone that is already $2^{24 \cdot 784} \approx 10^{5664}$ configurations — a number so vast that storing even one bit per image would require more matter than exists in the observable universe.

Yet the images we actually care about — natural photographs, handwritten digits, faces of real people, paragraphs of English prose — occupy only a **vanishingly small fraction** of that ambient space. Two observations explain why:

1. **Local correlation.** In a natural image, neighbouring pixels have similar colour. Random images do not. Therefore natural images live in a highly correlated, highly constrained subset of the ambient space.
2. **Low-entropy distributions.** When we further restrict attention to images that share some property (all contain giraffes; are all sevens; are all photographs of one particular human face), the valid points form tight **clusters** inside that already-constrained subset. In stochastic language, the distribution $p(\vec{x} \mid \text{class})$ is low-entropy, highly non-uniform.

These clusters are not scattered blobs. They tend to arrange themselves **along low-dimensional surfaces — manifolds** — embedded inside the high-dimensional ambient space.

The central idea of Chapter 14. The data of interest concentrates on (or near) a lower-dimensional manifold in the ambient input space. A good representation of the data throws away the ambient directions that are *off-manifold* (noise, background, irrelevant variation) and keeps only the *on-manifold* coordinates (the ones that index the structure we actually care about). That compressed representation is called the **latent vector** or **embedding**.

This chapter develops three progressively powerful techniques for discovering and using such manifolds:

Technique	Manifold shape	Training signal
PCA (§4)	best-fit hyperplane	variance maximization
Autoencoder (§6)	arbitrary curved hypersurface	reconstruction loss
Variational Autoencoder (Part 2)	curved hypersurface + smooth, regularized parameterization	reconstruction + KL divergence

2. Geometric View of Latent Spaces

Consider the mental picture of figure 14.1 in the text. A cloud of data points (each a high-dimensional vector) hovers around some surface. Two canonical sub-cases:

Case (a) — Planar manifold. The points cluster around a flat hyperplane. A single normal direction points *off* the plane, and two (or more) in-plane directions parameterize travel *along* the plane.

Case (b) — Curved manifold. The points cluster around a curved surface (think of a swiss roll or an S-curve). At each point on the surface we can still locally define "in-manifold" directions and "off-manifold" directions, but the in-manifold directions rotate as we move around.

2.1 Decomposing the Input Vector

Pick a training point $\vec{x}$. Project it onto the manifold to obtain the **in-manifold component** $\vec{x}_{\parallel}$. The leftover $\vec{x}_{\perp} = \vec{x} - \vec{x}_{\parallel}$ is **orthogonal** to the manifold. Any input vector decomposes as

$$\vec{x} = \vec{x}_{\parallel} + \vec{x}_{\perp}, \quad \vec{x}_{\perp} \perp \text{manifold at } \vec{x}_{\parallel}.$$

The **latent vector** $\vec{z}$ is the representation of $\vec{x}_{\parallel}$ in some coordinate system *intrinsic* to the manifold (e.g., arc-length along a curve, or (u, v) coordinates on a surface). Crucially, $\vec{z}$ lives in a space of **smaller dimension** than $\vec{x}$ — if the manifold is k -dimensional, $\vec{z} \in \mathbb{R}^k$ even though $\vec{x} \in \mathbb{R}^d$ with $k < d$.

2.2 Why Discarding $\vec{x}_{\perp}$ Is OK (and Often Helpful)

Under the working assumption that the manifold captures *everything that matters* for the task (giraffeness, digit-class, sentiment, ...), the orthogonal part $\vec{x}_{\perp}$ is by construction the variation *unrelated* to the task — background pixels, sensor noise, lighting artefacts, stylistic jitter.

Throwing it away:

- **Reduces dimensionality** (smaller, faster representations).
- **Denoises** the data (orthogonal component is noise by assumption).

- **Compresses** with minimal task-relevant information loss.
- **Reveals similarity** (two images that map to nearby $\vec{z}$ are semantically similar even if their pixel-level difference is large).

2.3 Projection Is Irreversible

A subtle but important point. The latent vector $\vec{z}$ records the **position on** the manifold, but not the **distance from** the manifold. If we try to reconstruct $\vec{x}$ from $\vec{z}$ alone we can only recover $\vec{x}_{||}$, not $\vec{x}$ itself. The decoder remembers, *on average*, where the manifold sits in the ambient space, which is enough to invert the encoder approximately — but the original orthogonal offset is **irrevocably lost**.

Reconstruction is approximate. **Exactness is traded for compression.**

2.4 Inferencing via Distance-to-Manifold

At test time, given an arbitrary new input $\vec{x}_{\text{new}}$, we can compute its distance from the learned manifold. A small distance says "this point looks like something in the training distribution"; a large distance says "this point is off-manifold — probably not a giraffe". This is how latent-space models produce **probabilistic** answers rather than hard classifications.

3. Generative vs. Discriminative Classifiers

Latent-space models are naturally **generative**. Understanding the contrast with **discriminative** models clarifies their advantages.

3.1 Definitions

	Discriminative	Generative
Models	$p(\text{class} \mid \vec{x})$ — decision boundary	$p(\vec{x} \mid \text{class})$ and/or $p(\vec{x})$ — density of the class
Output at test time	hard label (plus optional bounding box)	probability of belonging to a class
Failure mode	can overfit, adjusting the boundary to follow nooks and bends of the training data	bounded by a smooth density — structurally resistant to that kind of overfit
Can we sample new class instances?	no	yes

	Discriminative	Generative
Convert to the other?	← threshold a generative model's probability	not directly

The text's figures (14.2a–c) compare a smooth discriminative boundary, a zig-zag overfit discriminative boundary, and a generative heat-map. The heat-map elegantly avoids the question "which side of the boundary?" by assigning every point in the input space a probability of belonging to the class of interest.

3.2 Three Concrete Advantages of Generative Models

1. Smoother decision surfaces. Because a generative model parameterizes a smooth probability density, it cannot suddenly carve out an irregular protrusion to capture a single outlier. This is a form of **inductive bias**: when training data is scarce, generative models generalize better.

2. Extra diagnostic insight. Consider a "horse detector" trained discriminatively that also calls zebras horses. You'd need extra non-horse, non-zebra images to diagnose whether the model is "useless". A generative horse detector instead reports, e.g., $p(\text{horse}) = 0.92$ for horses vs 0.68 for zebras — revealing that the model *does* distinguish them but has set an inclusive threshold.

3. New instance generation. If you have $p(\vec{x})$, you can sample. Trained on Shakespeare, the model emits Shakespeare-like paragraphs. Trained on horse photos, it emits novel horse images. This is the root of modern generative AI.

Rule: Any generative classifier becomes a discriminative one by thresholding its probability output. The reverse conversion is not generally possible.

4. Benefits and Applications of Latent-Space Modelling

Before the mathematics, a compact survey of **why** this machinery is practically valuable.

4.1 Why we do it

- 1. Generative power.** All benefits of generative modelling (§3.2) apply automatically to any system built on top of a learned latent space.
- 2. Attention to what matters.** Redundant information that does not help the end goal is eliminated, and the system focuses on truly discriminative features. (The canonical cartoon: mugshot recognizers trained on subjects in front of the same wall will learn to represent subject-facial-features, not the wall.)

3. **Streamlined data representation.** The latent vector is a smaller, denser version of the input with no meaningful information lost.
4. **Noise elimination.** The low-variance orthogonal-to-latent-subspace component *is* the noise, and the projection removes it.
5. **Friendlier geometry for the downstream task.** A simple but illustrative example: points inside the unit disc $x^2 + y^2 \leq 1$ vs outside cannot be separated by a straight line in (x, y) , but the map $(x, y) \mapsto (r, \theta)$ turns the circle $x^2 + y^2 = 1$ into the straight line $r = 1$. A well-chosen latent space can turn nonlinear problems into linear ones.

4.2 Where it gets deployed

- **Generation of new images or text** — VAEs, GANs, diffusion models.
 - **Similarity retrieval.** Distance between latent vectors captures *perceptual* similarity better than pixel distance. A white-striped-on-black shirt and a black-striped-on-white shirt look different pixel-by-pixel but should be nearby in a good latent space (they are, perceptually, the same product with inverted colours).
 - **Compression.** A d -dimensional input maps to a k -dimensional latent ($k \ll d$) — a lossy but principled compressor.
 - **Denoising.** Encode then decode: the orthogonal component was thrown away in encoding, so the output is de-noised.
 - **Transfer learning.** Latent representations learned on a large dataset (ImageNet, LAION, all of Wikipedia) are useful initialization for downstream tasks with less data.
-

5. Linear Latent Spaces: PCA Revisited

Principal Component Analysis is the simplest latent-space model: the manifold is forced to be a **hyperplane** (a planar manifold of dimension k embedded in $\mathbb{R}^d$). PCA chooses the hyperplane that passes through the data mean and maximizes the variance retained after projection.

We spend a full section on PCA because (i) it is fast and closed-form, (ii) it establishes the vocabulary for nonlinear extensions, (iii) it is the exact limit of an autoencoder with no nonlinearities, and (iv) it yields the smallest reconstruction error among all *linear* projections to a subspace of a given size.

5.1 Setup

Let the data be the matrix $X \in \mathbb{R}^{n \times d}$, one row per sample, d features. Assume for now we want to reduce to $k = d - m$ dimensions (we drop the m least-variance directions).

Step 1 — Centre the data. Compute the mean $\vec{\mu} = \frac{1}{n} \sum_{i=1}^n \vec{x}^{(i)}$, then overwrite each row:

$$\vec{x}^{(i)} \leftarrow \vec{x}^{(i)} - \vec{\mu}, \quad i = 1, \dots, n.$$

We store $\vec{\mu}$ so we can un-centre later during reconstruction. From here on X denotes the **mean-subtracted** data matrix.

Step 2 — Covariance matrix. The matrix $X^\top X \in \mathbb{R}^{d \times d}$ is (up to a factor of n) the sample covariance of the features. Since it is real symmetric positive-semidefinite, it has an orthonormal eigenbasis.

Step 3 — Eigendecomposition. Write

$$X^\top X \vec{v}_j = \lambda_j \vec{v}_j, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0.$$

The pairs $(\lambda_j, \vec{v}_j)$ are the **principal values** and **principal vectors**. Stack the $\vec{v}_j$'s into the columns of

$$V = [\vec{v}_1 \mid \vec{v}_2 \mid \dots \mid \vec{v}_d] \in \mathbb{R}^{d \times d}, \quad V^\top V = I.$$

5.2 Why the Principal Vectors are the Right Directions

Suppose we project data onto a unit vector $\vec{u}$. The variance of the projected data is

$$\text{Var}(X\vec{u}) = \vec{u}^\top (X^\top X) \vec{u}.$$

Maximising this over unit vectors $\vec{u}$ is a classic Rayleigh-quotient problem whose answer is $\vec{u} = \vec{v}_1$ (the eigenvector of largest eigenvalue) with maximum value λ_1 . The next-best orthogonal direction is $\vec{v}_2$ with variance λ_2 , and so on. Therefore:

The principal vectors are the orthogonal directions of successively maximum variance. The principal values are the variances themselves.

If λ_j is small, the data barely varies along $\vec{v}_j$ — that direction is largely noise and can be discarded with tiny information loss.

5.3 SVD: The Preferred Algorithm

Computing $X^\top X$ and then its eigenvectors is numerically inferior to the **singular value decomposition**

$$X = U \Sigma V^\top, \quad U \in \mathbb{R}^{n \times d}, \Sigma = \text{diag}(s_1, \dots, s_d), V \in \mathbb{R}^{d \times d}.$$

The relationships are:

- $\lambda_j = s_j^2$ (principal values are squared singular values),
- The V from SVD is exactly the V of principal vectors,

- The columns of $U\Sigma$ give us the coordinates of each data point in the principal basis.

5.4 Encoding (Projection)

Drop the last m columns of V to obtain

$$V_{d-m} = [\vec{v}_1 \mid \cdots \mid \vec{v}_{d-m}] \in \mathbb{R}^{d \times (d-m)}.$$

The **latent representation** of the data is

$$Z = XV_{d-m} \in \mathbb{R}^{n \times (d-m)}$$

Each row of Z is a $(d - m)$ -dimensional latent vector — the "embedding" of the corresponding input point.

5.5 Decoding (Reconstruction)

To reconstruct, pad each latent vector with m zeros on the right (so that it is now d -dimensional in the principal basis), then rotate back to the original axes using $V^\top$, then add back the mean:

$$\tilde{X} = [Z \mid \mathbf{0}] V^\top + \mathbf{1} \vec{\mu}^\top.$$

Equivalently, in SVD form, $\tilde{X} = U\Sigma_{d-m}V^\top + \text{mean}$ where Σ_{d-m} zeroes out the smallest m singular values.

5.6 Reconstruction Loss and Optimality

The squared reconstruction error is

$$\|X - \tilde{X}\|_F^2 = \sum_{j=d-m+1}^d \lambda_j.$$

This equals the total variance that was thrown away — the sum of the smallest m principal values. The **Eckart–Young–Mirsky theorem** guarantees that among **all** rank- $(d - m)$ linear projections, PCA achieves the minimum reconstruction error. No linear projection can do better.

5.7 PyTorch Implementation

```
import torch

x_mean = X.mean(dim=0)
Xc      = X - x_mean
U, S, Vh = torch.linalg.svd(Xc, full_matrices=False)
V        = Vh.T                # principal vectors as columns
V_trunc  = V[:, :-1]           # drop the lowest-variance direction
```

```

Z          = Xc @ V_trunc          # (n, d-1) latent codes
Z_pad     = torch.cat([Z, torch.zeros(Z.shape[0], 1)], dim=1)
X_hat     = Z_pad @ V.T + x_mean  # reconstruction

```

5.8 Choosing How Many Components to Keep

Plot the cumulative variance explained

$$\text{CVE}(k) = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^d \lambda_j}$$

against k . The "elbow" of this curve is a common heuristic for the best latent dimensionality. Typical target: 95%–99% of variance preserved.

5.9 Worked Example: 3D Points Around the $X_0 = X_2$ Plane

Following the text's figure 14.3, suppose we generate 1,000 three-dimensional points clustered near the plane $X_0 = X_2$ with small Gaussian noise in the $X_0 - X_2$ direction.

- The covariance matrix has two large eigenvalues (variance in the plane) and one small eigenvalue (the thin noise direction normal to the plane).
- PCA with $k = 2$ throws away the normal direction, projecting every point onto the learned plane.
- Reconstruction is excellent because very little variance lived along the dropped direction.

(See the accompanying marimo notebook, §1, for a runnable PyTorch demo and 3D/2D plots.)

5.10 When PCA Fails

PCA projects to a **hyperplane**. If the data lies on a curved manifold, no plane fits well:

- **S-curve**. Data lives on a 2D curled sheet in 3D. A 2D PCA projection flattens the curl — points that were distant along the sheet get collapsed near each other.
- **Swiss roll**. Same pathology, more dramatic.
- **Sphere**. The entire data set is equidistant from every plane — reconstruction error is huge no matter which plane PCA picks.

To handle such data we need nonlinear projections. Enter autoencoders.

6. Autoencoders

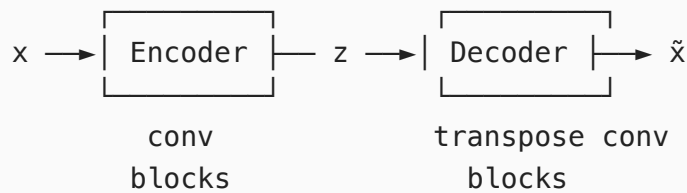
An **autoencoder (AE)** is a pair of neural networks that together learn an arbitrary nonlinear projection onto a latent manifold and its inverse.

6.1 Definition

$$\vec{z} = E(\vec{x}), \quad \tilde{\vec{x}} = D(\vec{z}), \quad \mathcal{L}_{\text{recon}}(\vec{x}) = \|\vec{x} - \tilde{\vec{x}}\|^2.$$

- **Encoder** E — a neural network that maps input $\vec{x} \in \mathbb{R}^d$ to a latent code $\vec{z} \in \mathbb{R}^{n_z}$ with $n_z \ll d$.
- **Decoder** D — a neural network that maps the latent code back to the input space, producing a reconstruction $\tilde{\vec{x}}$.
- The pair is trained **end to end** to minimize reconstruction loss.

Schematically,



The narrow point in the middle — the **bottleneck** — forces the network to compress. No task-irrelevant information can survive the squeeze.

6.2 Unsupervised Training

The "target" for an autoencoder is **the input itself**. No labels are needed. That makes autoencoders one of the canonical **self-supervised** / unsupervised methods. You can train one on every image you can scrape and then re-use the encoder as a pretrained feature extractor for downstream supervised tasks.

6.3 Convolutional Encoder for Image Data (Listing 14.2)

A standard choice for 28×28 MNIST inputs:

```
import torch.nn as nn

n_z = 8
input_image_size = (1, 28, 28)

conv_encoder = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size=3, stride=1, padding=1),
    nn.BatchNorm2d(16), nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),          # → 16 × 14 × 14
    nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1),
```

```

nn.BatchNorm2d(32), nn.ReLU(),
nn.MaxPool2d(kernel_size=2),          # → 32 × 7 × 7
nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),
nn.BatchNorm2d(64), nn.ReLU(),
nn.MaxPool2d(kernel_size=2),          # → 64 × 3 × 3
nn.Flatten(),                         # → 576
)
fc = nn.Linear(576, n_z)              # → n_z-dimensional latent

```

Every `MaxPool2d(2)` halves the spatial resolution, and every `Conv2d` expands the channel count. By the time we reach the linear layer the input has been squeezed from 784 dimensions down to 576 spatial features, then down to n_z .

6.4 Convolutional Decoder (Listing 14.3)

The decoder mirrors the encoder, replacing `Conv2d` → `MaxPool` with `ConvTranspose2d` to re-inflate the spatial resolution:

```

conv_decoder = nn.Sequential(
    # reshape n_z → 64 × 3 × 3 via a linear then view
    nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2, padding=0),
    nn.BatchNorm2d(32), nn.ReLU(),          # → 32 × 7 × 7
    nn.ConvTranspose2d(32, 16, kernel_size=3, stride=2, padding=1,
output_padding=1),
    nn.BatchNorm2d(16), nn.ReLU(),          # → 16 × 14 × 14
    nn.ConvTranspose2d(16, 1, kernel_size=3, stride=2, padding=1,
output_padding=1),
    nn.Sigmoid(),                          # → 1 × 28 × 28 in [0, 1]
)

```

Transposed convolution (Chapter 10) is the "learnable upsampling" operation. `Sigmoid` at the end squashes the output to $[0, 1]$, matching the pixel range of MNIST.

6.5 Training Loop

```

import torch.nn.functional as F

def step(x, encoder, fc, decoder, optim):
    z      = fc(encoder(x))
    x_hat  = decoder(z.view(-1, 64, 3, 3))
    loss   = F.mse_loss(x_hat, x)
    optim.zero_grad(); loss.backward(); optim.step()
    return loss

```

Typical choices: Adam optimizer, learning rate 10^{-3} , batch size 128, 10–20 epochs for a rough MNIST autoencoder.

6.6 Asymmetric Architectures

The encoder and decoder need **not** be mirror images. It is common to use a deeper decoder than encoder when the reconstruction task is harder than the encoding task, and the reverse for fast deployment scenarios. The only constraint is that the output of the decoder has the same shape as the input.

6.7 Autoencoder vs PCA

Suppose both the encoder and decoder are single linear layers with no activation functions and no biases:

$$E(\vec{x}) = W_E \vec{x}, \quad D(\vec{z}) = W_D \vec{z}.$$

Then the reconstruction is $\tilde{\vec{x}} = W_D W_E \vec{x}$. Minimising $\|\vec{x} - W_D W_E \vec{x}\|^2$ over rank- k matrices $W_D W_E$ recovers (up to a basis change) the PCA projection. In this limit **autoencoder = PCA**.

The instant we add a ReLU (or any nonlinearity) anywhere in the encoder/decoder, the class of realizable projections expands from hyperplanes to **arbitrary curved hypersurfaces**. This is the qualitative leap from PCA to a real autoencoder.

6.8 What Information Is Stored Where?

This is the most useful mental model:

Information about	Lives in
The specific input $\vec{x}$	the latent vector $\vec{z}$
The shape and location of the manifold in the ambient space	the decoder weights
How to project an arbitrary ambient point onto the manifold	the encoder weights

The decoder's weights are, collectively, an average memory of "where the data manifold sits". The encoder's weights know how to project any new point onto that manifold. Neither network knows anything about any individual training example after training — that information is carried by the latent code.

7. Limits of a Plain Autoencoder

A plain autoencoder has a serious structural weakness: **reconstruction loss does not uniquely determine the latent space**. Two radically different latent spaces can achieve equally low reconstruction error.

7.1 The Zig-Zag Pathology

Imagine 2D input data with a linear trend. We want to compress to a 1D latent space. Two candidate 1D manifolds both fit the training data:

- A **smooth straight line** that passes through the data cloud.
- A **zig-zag curve** that threads through every single training point exactly.

Both achieve **zero** reconstruction error on the training set. But they are not equally good:

	Smooth line	Zig-zag
Reconstruction on <i>training</i> points	0	0
Reconstruction on <i>held-out</i> points	small	huge
Distance between two nearby inputs measured along the manifold	proportional to input distance	wildly distorted
Manifold length	short	long

The zig-zag has essentially **memorised** the training set. It overfits despite having low training loss. This is analogous to a polynomial regression model with degree equal to the number of training points — perfect training fit, disastrous test behaviour.

7.2 Why This Matters

A deep, overparameterised autoencoder can easily learn a zig-zag-like latent manifold. The danger manifests in three concrete ways:

1. **Interpolation fails.** Sampling a latent vector halfway between two training codes decodes to garbage because the straight segment between them in latent space may pass through long detours in input space.
2. **Generation fails.** Sampling latent vectors from any distribution that doesn't already match the (unknown, irregular) training distribution yields garbage outputs.
3. **Similarity fails.** Euclidean distance in a zig-zag latent space has no relation to perceptual distance in input space.

7.3 What We Want: The Minimum Description Length Principle

A good latent manifold:

1. **Fits** the training data well (low reconstruction loss).
2. Is **as short / flat as possible** (few twists and turns).
3. Is **compact** (the training data's latent codes do not spread indefinitely far apart).
4. Is **continuous** — small changes in $\vec{z}$ yield small changes in $\tilde{\vec{x}}$.

Properties 2–4 echo the **Minimum Description Length (MDL) principle**: among models that fit the data, prefer the one that requires the fewest bits to describe. They also echo **Occam's Razor**: prefer the simpler of two equally predictive models.

7.4 How to Impose These Properties

Two complementary strategies:

1. **Regularization**. Add an explicit loss that penalizes undesirable latent-space structure (long manifolds, spread codes, large encoder weights).
2. **Probabilistic modelling**. Model the latent space as a probability distribution from a simple family (e.g., Gaussian) and demand that the distribution stay close to a compact target (e.g., $\mathcal{N}(0, I)$) via a divergence penalty.

Strategy 2, combined with a clever trick for keeping the network differentiable, yields **variational autoencoders** — the subject of Lecture 23.

8. Summary and Preview

What we covered

- Data of interest concentrates on low-dimensional **manifolds** inside high-dimensional ambient spaces.
- A **latent vector** is the in-manifold coordinate of a data point. The orthogonal component is discarded as noise.
- **Generative models** describe the data density $p(\vec{x})$. They are smoother, more diagnostic, and can sample new instances — unlike discriminative models that only draw decision boundaries.
- **PCA** is the simplest latent-space model: best-fit hyperplane via SVD. Closed-form, fast, optimal among **linear** projections; fails on curved manifolds.
- **Autoencoders** replace PCA's hyperplane with an arbitrary nonlinear hypersurface via neural-network encoder / decoder pairs trained by reconstruction loss.
- Plain autoencoders have no structural constraint on the latent space — they can learn zig-zag manifolds that overfit, interpolate poorly, and cannot generate.

What comes next (Lecture 23)

- Make the latent space **smooth, compact, and continuous** by modelling it as a probability distribution.
 - The **Variational Autoencoder (VAE)** — a stochastic encoder that emits the parameters of a distribution, a KL-divergence regularizer pulling that distribution toward $\mathcal{N}(0, I)$, and the **reparameterization trick** that keeps the whole system differentiable.
 - The **Evidence Lower BOund (ELBO)** — the variational objective that unifies reconstruction loss and KL regularization into a principled lower bound on $\log p(\vec{x})$.
 - Side-by-side comparison of AE and VAE latent spaces on MNIST, showing why VAEs can generate new digits and AEs cannot.
-

9. Companion Materials (Lecture 22 portion)

- **Marimo notebook** — `week-13/code/IME775_Ch14_Autoencoders_VAE_marimo.py` .
Relevant sections for this lecture:
 - §1 PCA in PyTorch (3D $\rightarrow$ 2D)
 - §2 S-curve: where PCA fails
 - §3 Convolutional autoencoder on MNIST
 - **Interactive HTML visualizations** — `week-13/visualizations/`
 - `ae-vs-vae.html` — preview of how the KL term will compact the latent space (set $\beta = 0$ to see pure autoencoder behaviour).
-

Reference: Krishnendu Chaudhury, *Math and Architectures of Deep Learning*, Chapter 14 §§14.1–14.6.